

Projektwissen festhalten — ohne daran zu denken

Zwei Claude-Skills, die zusammenarbeiten. Diese Anleitung zeigt, was sie tun, wie du sie einrichtest und was danach im Alltag passiert.

Das Problem: Ihr klärt etwas Wichtiges im Chat — warum ihr Weg A statt B geht, wie ein Teil aufgebaut ist. Zwei Wochen später weiß es niemand mehr, und der nächste Chat fängt bei null an.

Die Lösung: Claude schreibt es in Dateien im Projekt — automatisch, im richtigen Moment. Jeder neue Chat und jede Kollegin liest es von dort.

Zwei Begriffe vorab: Ein **Skill** ist ein fertiger Arbeitsablauf, den du mit einem Schrägstrich aufrufst, z. B. `/decision-records`. Ein **Hook** ist eine automatische Regel, die in bestimmten Momenten von selbst anspringt — du musst nichts tippen.

Die zwei Skills — und warum es zwei sind

`/decision-records`

„Warum haben wir das so entschieden?“

- Diese Antwort steht **nirgends im Code** — nur in deinem Kopf.
- Er **nimmt an, was naheliegt**, und schreibt die Annahme sichtbar dazu — statt dich mit Fragen aufzuhalten.
- Schreibt das Ergebnis mit Datum und Namen fest.

`/doku-update-sync`

„Stimmt die Doku noch mit dem Projekt überein?“

- Das **sieht Claude selbst** an euren Änderungen.
- Deshalb **fragt er nicht**, sondern schreibt es still mit.
- Sorgt außerdem dafür, dass das alles von allein passiert.

Wie sie zusammenarbeiten

`/decision-records`

beim ersten Mal im Projekt

richtet `/doku-update-sync` ein

Automatik, Regeln, Doku-Dateien

Ein Befehl — beide Skills sind startklar.

Automatik merkt: da war was

nach einer Aufgabe, am Session-Ende

Claude schreibt still mit

was er selbst sehen kann

Du merkst davon normalerweise nichts.

Das „Warum“ fehlt

Claude kann es nicht ableiten

`/decision-records` hält es fest

nimmt an, was naheliegt

Gefragt wirst du nur, wenn eine falsche Annahme teuer wäre.

1 Einmalig: einen Skill aktivieren

Die Skills liegen bereits in unserer Organisation. Jede Person schaltet einmal im eigenen Profil frei: **claud.ai** → **Einstellungen** → **Skills** → `decision-records` einschalten. **Das genügt** — den zweiten Skill holt dieser sich später selbst dazu.

Test: In Claude Code `/` tippen. Steht `decision-records` in der Liste, bist du fertig.

2 Pro Projekt: einmal einrichten

Claude Code im Projektordner starten und eingeben:

```
/decision-records
```

Claude sieht sich das Projekt an und **leitet das meiste selbst ab**: welche Doku dieses Projekt braucht und was als wichtig gilt (z. B. „Änderungen an den Workflows ja, Notizordner nein“). Gefragt wirst du nur zu den zwei Dingen, die sich schlecht zurücknehmen lassen: ob die Automatik einen `git push` wirklich anhalten darf, und ob er sie einschalten darf. Am Ende zeigt er alles und fragt, ob er es speichern darf. **Nichts passiert ohne dein Ja.**

Ist der Ordner ganz neu, fragt er vorweg, ob er ihn als Projekt anlegen darf (`git init`) — sag ja, sonst erkennt die Automatik später keine Änderungen. Den zweiten Skill lädt er sich selbst dazu und legt beide mit ins Projekt.

Der Alltag danach

Kurz gesagt: meistens merkst du gar nichts. Genau das ist der Punkt.

Wann die Automatik nachschaut

MOMENT	WAS PASSIERT
Aufgabe erledigt	Still. Der Moment, in dem das Wissen am frischesten ist.
Am Ende der Session	Still. Damit nichts verloren geht, bevor du zumachst.
Vor <code>git push</code>	Fehlt Doku, ergänzt Claude sie sofort selbst — danach läuft der Push durch.

Wichtig: Gibt es nichts zu dokumentieren, sagt die Automatik **gar nichts**. Kein „alles in Ordnung“, keine Rückfrage. Sie meldet sich nur, wenn sie wirklich etwas beizutragen hat.

Wann Claude fragt — und wann nicht

Er schreibt allein

Alles, was er im Projekt sehen kann: ein neuer Baustein, eine neue Bibliothek, ein geändertes Datenmodell, ein überholter Stand. Auch beim **Warum** schreibt er, was aus eurem Gespräch naheliegt — und vermerkt dabei, dass es eine Annahme ist.

Er fragt nach

Nur wenn eine falsche Annahme teuer wäre: wenn er eine frühere Entscheidung *umschreiben* würde, wenn etwas dem bisherigen Stand widerspricht, oder wenn er an der Automatik selbst etwas ändert.

Eine Annahme, die nicht stimmt, kostet dich einen Satz im Chat („die Begründung war eine andere: …“) — Claude zieht den Eintrag nach. Das ist Absicht: lieber eine korrigierbare Annahme im Eintrag als eine Rückfrage, die dich aus der Arbeit reißt.

Wenn der Push angehalten wird — du musst nichts tun

Dann fehlt Doku zu etwas, das du geändert hast. Claude fragt dich **nicht**, ob er das nachholen soll — er legt sofort los, schreibt die Doku, speichert sie und pusht dann normal weiter. Meistens dauert das unter einer Minute, und du siehst nur eine kurze Notiz, was ergänzt wurde.

```
# Was du eingibst
git push

# Was von allein passiert
Doku wird ergänzt → gespeichert → Push läuft durch
```

Nur falls eine Änderung wirklich gar keine Doku braucht, gibt es eine bewusste Abkürzung: `DOC_SYNC_SKIP=1 git push`. Die wählst du selbst — Claude benutzt sie nie von sich aus.

Der eigentliche Gewinn: jeder neue Chat kennt das Projekt

Beim Einrichten trägt Claude einen Verweis in die Datei `CLAUDE.md` ein — die einzige Datei, die er in **jedem neuen Chat automatisch liest**. Von dort findet er die Entscheidungen und den aktuellen Stand selbst.

Du musst also **nicht im selben Chat bleiben** und nichts zweimal erklären. Das gilt auch für alle anderen im Team und auch noch in einem halben Jahr.

Nervt die Automatik? Dann ist eingestellt, dass zu viel als wichtig gilt — nicht die Idee ist falsch. Sag es Claude einfach im Chat („der Sync meldet sich zu oft wegen X“), er passt die Einstellung an. Jeder Moment lässt sich auch einzeln abschalten.

Der kürzeste Weg zum Merken: Du brauchst nur `/decision-records`, wenn ihr etwas entschieden habt — alles Übrige läuft von allein. Und wenn du unsicher bist, frag Claude im Projekt: „Wie ist der Doku-Sync hier eingestellt?“